COLLEGE ERP

# Finance Module

Final Database Schema — 10 Tables with Examples

Modular Fee Billing · Voteheads · Receipts · Reversals · Refunds

| | |
|---|---|
| **Tables** | 10 |
| **Billing Unit** | Module Enrollment |
| **Fee Resolution** | By enrollment date — not intake year |
| **Distribution** | Computed on receipt generation |
| **Reversals** | Immutable — separate rows, never deleted |
| **Audience** | Development Team |

# Contents


| | | |
|---|---|---|
| 01 | `modules` | Course codes, names, levels, module numbers |
| 02 | `voteheads` | Fee categories with priority, one-time and refundable flags |
| 03 | `fee_structures` | Named fee configs per module, category, and date range |
| 04 | `fee_structure_lines` | Votehead amounts within each fee structure |
| 05 | `module_enrollments` | Billing trigger — locks fee_structure_id at enrollment |
| 06 | `fee_obligations` | Live balance per enrollment — total_paid, total_balance |
| 07 | `payments` | Immutable payment records including reversals |
| 08 | `payment_distributions` | Votehead breakdown — written at receipt generation |
| 09 | `refunds` | Programme-completion refunds (caution etc.) |
| 10 | `audit_log` | Every financial action with before/after snapshots |

## 1 modules

Owns the definition of every module offered by the college. Course code, name, level, and module number all live here. Both **fee_structures** and **module_enrollments** reference this table via FK — eliminating duplicate course/level/module columns everywhere else.

SQL: modules

```
CREATE TABLE modules (
    id           VARCHAR(20) PRIMARY KEY,
    course_code  VARCHAR(20) NOT NULL,
    course_name  VARCHAR(150) NOT NULL,
    level        INT NOT NULL,
    module_num   INT NOT NULL,
    is_active    BOOLEAN DEFAULT TRUE,
    created_at   TIMESTAMP DEFAULT NOW(),
    UNIQUE(course_code, level, module_num)
);
```

**Example data:**

| id | course_code | course_name | level | module_num | is_active |
|---|---|---|---|---|---|
| MOD001 | DICT | Diploma in ICT | 4 | 1 | true |
| MOD002 | DICT | Diploma in ICT | 4 | 2 | true |
| MOD003 | DICT | Diploma in ICT | 4 | 3 | true |
| MOD004 | NBC | Diploma in Business & Commerce | 4 | 1 | true |
| MOD005 | NBC | Diploma in Business & Commerce | 4 | 2 | true |
| MOD006 | AHBT | Diploma in Hospitality | 4 | 1 | true |
| MOD007 | ICTP | Computer Packages | 3 | 1 | true |
| MOD008 | DBT | Diploma in Building Tech | 4 | 1 | true |

**Note:** UNIQUE(course_code, level, module_num) prevents duplicate module definitions. course_code is a reference to the Academics microservice — stored here as a plain string, no DB-level FK since it lives in a separate service.

## 2 voteheads

Institution-level master list of fee categories. The finance admin owns this entirely — add, remove, rename, reorder at any time. **priority_order** drives the waterfall distribution engine at receipt generation. **is_one_time** means the fee is charged only once across all a student's enrollments. **is_refundable** flags voteheads that can be returned to the student on programme completion.

SQL: voteheads

```
CREATE TABLE voteheads (
    id              VARCHAR(20) PRIMARY KEY,
    name            VARCHAR(100) NOT NULL UNIQUE,
    description     VARCHAR(200),
    is_one_time     BOOLEAN DEFAULT FALSE,
    is_refundable   BOOLEAN DEFAULT FALSE,
    priority_order  INT NOT NULL UNIQUE,
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | name | is_one_time | is_refundable | priority | is_active |
|---|---|---|---|---|---|
| VH001 | Admission Fee | true | false | 1 | true |
| VH002 | ID Card | true | false | 2 | true |
| VH003 | Caution Fee | true | true | 3 | true |
| VH004 | Tuition | false | false | 4 | true |
| VH005 | PE | false | false | 5 | true |
| VH006 | RMI | false | false | 6 | true |
| VH007 | EWC | false | false | 7 | true |
| VH008 | Medical & Insurance | false | false | 8 | true |
| VH009 | Activity Fee | false | false | 9 | true |
| VH010 | Attachment Fee | false | false | 10 | true |
| VH011 | Examination Fee | false | false | 11 | true |
| VH012 | LTT | false | false | 12 | true |

## 3 fee_structures

One named fee configuration per module + student category + date range. **module_id** replaces the old course_id/level/module_index columns — all that context now lives in the modules table. **valid_from / valid_to** control which structure applies on any given enrollment date. valid_to = NULL means currently active with no end date. The enrollment date — not the student's original intake year — determines which structure applies.

SQL: fee_structures

```
CREATE TABLE fee_structures (
    id                  VARCHAR(20) PRIMARY KEY,
    name                VARCHAR(150) NOT NULL,
    module_id           VARCHAR(20) NOT NULL REFERENCES modules(id),
    student_category    VARCHAR(30) NOT NULL,
    -- regular | nita | kuccps | employer | short_course
    valid_from          DATE NOT NULL,
    valid_to            DATE,          -- NULL = no end date, currently active
    total_amount        NUMERIC(10,2) NOT NULL,
    is_active           BOOLEAN DEFAULT TRUE,
    created_by          VARCHAR(20) NOT NULL,  -- user_id from Auth service
    created_at          TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | module_id | student_category | valid_from | valid_to | total_amount |
|---|---|---|---|---|---|
| FS001 | MOD001 | regular | 2025-01-01 | 2025-12-31 | 37,000.00 |
| FS002 | MOD001 | regular | 2026-01-01 | 2026-12-31 | 40,000.00 |
| FS003 | MOD002 | regular | 2025-01-01 | 2029-12-31 | 38,500.00 |
| FS004 | MOD002 | regular | 2030-01-01 | NULL | 52,000.00 |
| FS005 | MOD004 | nita | 2025-01-01 | NULL | 36,000.00 |
| FS006 | MOD007 | short_course | 2025-01-01 | NULL | 4,500.00 |
| FS007 | MOD001 | kuccps | 2025-01-01 | 2025-12-31 | 37,000.00 |

**Fee structure lookup query** — run when enrolling a student:

QUERY: resolve fee structure at enrollment

```
SELECT fs.id
FROM   fee_structures fs
WHERE  fs.module_id        = :module_id
AND    fs.student_category = :student_category
AND    fs.valid_from      <= :enrollment_date
AND    (fs.valid_to       >= :enrollment_date OR fs.valid_to IS NULL)
AND    fs.is_active        = TRUE
LIMIT  1;
```

## 4 fee_structure_lines

The individual votehead entries inside each fee structure. One row per votehead per structure. The UNIQUE constraint prevents the same votehead appearing twice in one structure. At receipt generation, these lines are loaded in priority_order and used to run the waterfall distribution.

SQL: fee_structure_lines

```
CREATE TABLE fee_structure_lines (
    id                  VARCHAR(20) PRIMARY KEY,
    fee_structure_id    VARCHAR(20) NOT NULL REFERENCES fee_structures(id),
    votehead_id         VARCHAR(20) NOT NULL REFERENCES voteheads(id),
    amount              NUMERIC(10,2) NOT NULL,
    priority_order      INT NOT NULL,
    UNIQUE(fee_structure_id, votehead_id)
);
```

**FS001 — DICT L4 Mod1 Regular 2025 (total 37,000):**

| id | fee_structure_id | votehead_id | amount | priority_order |
|---|---|---|---|---|
| FSL001 | FS001 | VH001 | 500.00 | 1 |
| FSL002 | FS001 | VH002 | 500.00 | 2 |
| FSL003 | FS001 | VH003 | 1,000.00 | 3 |
| FSL004 | FS001 | VH004 | 15,000.00 | 4 |
| FSL005 | FS001 | VH005 | 5,250.00 | 5 |
| FSL006 | FS001 | VH006 | 2,000.00 | 6 |
| FSL007 | FS001 | VH007 | 1,870.00 | 7 |
| FSL008 | FS001 | VH008 | 2,400.00 | 8 |
| FSL009 | FS001 | VH009 | 1,430.00 | 9 |
| FSL010 | FS001 | VH010 | 2,550.00 | 10 |
| FSL011 | FS001 | VH011 | 3,000.00 | 11 |

**FS005 — NBC NITA 2025 (total 36,000) — different voteheads, LTT included:**

| id | fee_structure_id | votehead_id | amount | priority_order |
|---|---|---|---|---|
| FSL020 | FS005 | VH001 | 500.00 | 1 |
| FSL021 | FS005 | VH003 | 1,000.00 | 2 |
| FSL022 | FS005 | VH004 | 14,500.00 | 3 |
| FSL023 | FS005 | VH005 | 5,250.00 | 4 |
| FSL024 | FS005 | VH008 | 2,000.00 | 5 |
| FSL025 | FS005 | VH009 | 2,400.00 | 6 |

| id | fee_structure_id | votehead_id | amount | priority_order |
|---|---|---|---|---|
| FSL026 | FS005 | VH011 | 3,000.00 | 7 |
| FSL027 | FS005 | VH012 | 5,350.00 | 8 |

**FS006 — Packages short course (total 4,500) — Tuition only:**

| id | fee_structure_id | votehead_id | amount | priority_order |
|---|---|---|---|---|
| FSL030 | FS006 | VH004 | 4,500.00 | 1 |

## 5 module_enrollments

The billing trigger. Created when a student is registered for a module. **fee_structure_id is locked at enrollment time and never changes.** If the finance admin updates a fee structure later, existing enrollments are unaffected. One enrollment automatically generates one fee_obligation.

SQL: module_enrollments

```
CREATE TABLE module_enrollments (
    id                  VARCHAR(20) PRIMARY KEY,
    student_id          VARCHAR(20) NOT NULL,   -- from Admissions service
    module_id           VARCHAR(20) NOT NULL REFERENCES modules(id),
    fee_structure_id    VARCHAR(20) NOT NULL REFERENCES fee_structures(id),
    enrollment_date     DATE NOT NULL,
    enrolled_by         VARCHAR(20) NOT NULL,   -- user_id from Auth service
    status              VARCHAR(20) DEFAULT 'active',
    -- active | completed | withdrawn | deferred
    notes               TEXT,
    created_at          TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | student_id | module_id | fee_structure_id | enrollment_date | status |
|---|---|---|---|---|---|
| ENR001 | S12345 | MOD001 | FS001 | 2025-01-06 | completed |
| ENR002 | S12345 | MOD007 | FS006 | 2025-01-06 | completed |
| ENR003 | S12346 | MOD004 | FS005 | 2025-01-08 | active |
| ENR004 | S12347 | MOD001 | FS007 | 2025-01-10 | completed |
| ENR005 | S12345 | MOD002 | FS004 | 2030-03-15 | active |

**Note:** ENR001 and ENR002 show S12345 enrolled in two modules simultaneously in 2025 — DICT Mod1 and Packages. Two separate obligations created. ENR005 shows the same student returning in 2030 for DICT Mod2 — FS004 applies (52,000), not FS001 (37,000). Enrollment date drives fee resolution.

## 6 fee_obligations

Auto-created when a module_enrollment is created. One obligation per enrollment (enforced by UNIQUE on enrollment_id). This is the live running balance. **total_paid** and **total_balance** are updated atomically every time a payment is posted — no other tables are written to at payment time. Query this table alone to answer "does this student owe money?" with no joins.

SQL: fee_obligations

```
CREATE TABLE fee_obligations (
    id              VARCHAR(20) PRIMARY KEY,
    enrollment_id   VARCHAR(20) NOT NULL UNIQUE REFERENCES module_enrollments(id),
    student_id      VARCHAR(20) NOT NULL,
    total_amount    NUMERIC(10,2) NOT NULL,   -- frozen from fee_structure at enrollment
    total_paid      NUMERIC(10,2) NOT NULL DEFAULT 0,
    total_balance   NUMERIC(10,2) NOT NULL,   -- always total_amount - total_paid
    status          VARCHAR(20) NOT NULL DEFAULT 'unpaid',
    -- unpaid | partial | cleared | overpaid
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | enrollment_id | student_id | total_amount | total_paid | total_balance | status |
|---|---|---|---|---|---|---|
| OBL001 | ENR001 | S12345 | 37,000.00 | 15,000.00 | 22,000.00 | partial |
| OBL002 | ENR002 | S12345 | 4,500.00 | 4,500.00 | 0.00 | cleared |
| OBL003 | ENR003 | S12346 | 36,000.00 | 27,000.00 | 9,000.00 | partial |
| OBL004 | ENR004 | S12347 | 37,000.00 | 37,000.00 | 0.00 | cleared |
| OBL005 | ENR005 | S12345 | 52,000.00 | 0.00 | 52,000.00 | unpaid |

**What happens on payment posting:**

LOGIC: payment posting

```
-- Step 1: Insert payment row
INSERT INTO payments (...) VALUES (...);

-- Step 2: Update obligation balance (only this happens at payment time)
UPDATE fee_obligations
SET    total_paid    = total_paid + :amount,
       total_balance = total_balance - :amount,
       status        = CASE
                           WHEN total_balance - :amount <= 0 THEN 'cleared'
                           WHEN total_paid + :amount > 0     THEN 'partial'
                           ELSE 'unpaid'
                       END,
       updated_at    = NOW()
WHERE  id = :obligation_id;

-- payment_distributions are NOT written here.
-- They are written only when a receipt is generated.
```

## 7 payments

One row per payment transaction. **Immutable after creation — never edited or deleted.** Reversals are new rows with is_reversal = true and reversal_of pointing to the original. receipt_number is sequential, system-generated, and unique forever. Reversals use a different series prefix (RVS- instead of RCP-).

SQL: payments

```
CREATE TABLE payments (
    id              VARCHAR(20) PRIMARY KEY,
    student_id      VARCHAR(20) NOT NULL,
    receipt_number  VARCHAR(30) NOT NULL UNIQUE,
    total_amount    NUMERIC(10,2) NOT NULL,
    payment_mode    VARCHAR(20) NOT NULL,
    -- cash | mpesa | bank_transfer | cheque
    transaction_ref VARCHAR(100),             -- M-Pesa code, bank ref, cheque no.
    funding_source  VARCHAR(30) NOT NULL,
    -- self | kuccps | nita | employer | cdf_bursary | county_bursary | other
    sponsor_name    VARCHAR(100),             -- if employer or bursary
    received_by     VARCHAR(20) NOT NULL,     -- user_id from Auth service
    is_reversal     BOOLEAN DEFAULT FALSE,
    reversal_of     VARCHAR(20) REFERENCES payments(id),
    payment_date    DATE NOT NULL,
    notes           TEXT,
    created_at      TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | receipt_no | student_id | amount | mode | funding_source | is_reversal | reversal_of |
|---|---|---|---|---|---|---|---|
| PMT001 | RCP-2025-0001 | S12345 | 5,000.00 | mpesa | self | false | NULL |
| PMT002 | RCP-2025-0002 | S12345 | 10,000.00 | cash | self | false | NULL |
| PMT003 | RCP-2025-0003 | S12346 | 27,000.00 | bank_transfer | nita | false | NULL |
| PMT004 | RCP-2025-0004 | S12347 | 37,000.00 | bank_transfer | kuccps | false | NULL |
| PMT005 | RCP-2025-0005 | S12345 | 4,500.00 | cash | self | false | NULL |
| PMT006 | RVS-2025-0001 | S12345 | 5,000.00 | mpesa | self | true | PMT001 |

**Note:** PMT006 is the reversal of PMT001. Both rows are permanent. PMT001 is not edited. The net effect on OBL001 after PMT001 + PMT002 + PMT006 = 5,000 + 10,000 - 5,000 = 10,000 paid.

## 8 payment_distributions

Written at **receipt generation time**, not at payment posting time. When a receipt is requested, the system replays all payments for the obligation through the priority waterfall from scratch, then writes these rows as the permanent record of that receipt. Reprinting reads existing rows — no recalculation.

SQL: payment_distributions

```
CREATE TABLE payment_distributions (
    id              VARCHAR(20) PRIMARY KEY,
    payment_id      VARCHAR(20) NOT NULL REFERENCES payments(id),
    obligation_id   VARCHAR(20) NOT NULL REFERENCES fee_obligations(id),
    votehead_id     VARCHAR(20) NOT NULL REFERENCES voteheads(id),
    amount_applied  NUMERIC(10,2) NOT NULL
);
```

**Waterfall calculation logic (runs at receipt generation):**

LOGIC: waterfall at receipt generation

```
-- net_paid = SUM of all payments for this obligation (including reversals)
net_paid = SELECT SUM(p.total_amount)
           FROM payments p
           JOIN payment_distributions pd ON pd.payment_id = p.id
           WHERE pd.obligation_id = :obligation_id;

-- then run the waterfall:
remaining = net_paid
FOR EACH line IN fee_structure_lines WHERE fee_structure_id = :fs_id
                                    ORDER BY priority_order ASC:
    fill = MIN(line.amount, remaining)
    IF fill > 0:
        INSERT payment_distribution(payment_id, obligation_id, votehead_id, fill)
        remaining -= fill
    IF remaining == 0: STOP
```

**Example — OBL001, net paid = 10,000 (PMT001 5k + PMT002 10k - PMT006 reversal 5k):**

EXAMPLE: waterfall trace for OBL001

```
Waterfall for FS001 (DICT Mod1 Regular):

  Admission   500    -> fill 500,   remaining 9500
  ID Card     500    -> fill 500,   remaining 9000
  Caution    1000    -> fill 1000,  remaining 8000
  Tuition   15000    -> fill 8000,  remaining 0 -> STOP
  PE, RMI, EWC ...   -> skipped, no money left
```

**Resulting payment_distributions rows:**

| id | payment_id | obligation_id | votehead_id | amount_applied |
|---|---|---|---|---|
| DST001 | PMT001* | OBL001 | VH001 | 500.00 |

| id | payment_id | obligation_id | votehead_id | amount_applied |
|---|---|---|---|---|
| DST002 | PMT001* | OBL001 | VH002 | 500.00 |
| DST003 | PMT001* | OBL001 | VH003 | 1,000.00 |
| DST004 | PMT001* | OBL001 | VH004 | 8,000.00 |

**The breakdown query — used for receipts, student statements, dashboard:**

QUERY: votehead breakdown for receipt

```
SELECT
    v.name                                          AS votehead,
    fsl.amount                                      AS amount_due,
    COALESCE(SUM(pd.amount_applied), 0)             AS amount_paid,
    fsl.amount - COALESCE(SUM(pd.amount_applied), 0)   AS amount_balance,
    CASE
        WHEN COALESCE(SUM(pd.amount_applied), 0) = 0           THEN 'unpaid'
        WHEN COALESCE(SUM(pd.amount_applied), 0) >= fsl.amount THEN 'cleared'
        ELSE 'partial'
    END AS status
FROM fee_structure_lines fsl
JOIN voteheads v         ON v.id   = fsl.votehead_id
LEFT JOIN payment_distributions pd
                         ON pd.obligation_id = :obligation_id
                        AND pd.votehead_id   = fsl.votehead_id
WHERE fsl.fee_structure_id = :fee_structure_id
GROUP BY v.name, fsl.amount, fsl.priority_order, fsl.votehead_id
ORDER BY fsl.priority_order ASC;
```

**Result for OBL001 after net 10,000 paid:**

| votehead | amount_due | amount_paid | amount_balance | status |
|---|---|---|---|---|
| Admission Fee | 500.00 | 500.00 | 0.00 | cleared |
| ID Card | 500.00 | 500.00 | 0.00 | cleared |
| Caution Fee | 1,000.00 | 1,000.00 | 0.00 | cleared |
| Tuition | 15,000.00 | 8,000.00 | 7,000.00 | partial |
| PE | 5,250.00 | 0.00 | 5,250.00 | unpaid |
| RMI | 2,000.00 | 0.00 | 2,000.00 | unpaid |
| EWC | 1,870.00 | 0.00 | 1,870.00 | unpaid |
| Medical & Insurance | 2,400.00 | 0.00 | 2,400.00 | unpaid |
| Activity Fee | 1,430.00 | 0.00 | 1,430.00 | unpaid |
| Attachment Fee | 2,550.00 | 0.00 | 2,550.00 | unpaid |
| Examination Fee | 3,000.00 | 0.00 | 3,000.00 | unpaid |

## 9 refunds

Programme-completion refunds — distinct from reversals. A refund is money going back to a student after completing their programme (e.g. Caution Fee). A reversal corrects a posting error. Refunds have their own receipt number series (RCF-). Both authorised_by and processed_by are recorded for dual-control compliance.

SQL: refunds

```
CREATE TABLE refunds (
    id              VARCHAR(20) PRIMARY KEY,
    student_id      VARCHAR(20) NOT NULL,
    obligation_id   VARCHAR(20) NOT NULL REFERENCES fee_obligations(id),
    votehead_id     VARCHAR(20) NOT NULL REFERENCES voteheads(id),
    receipt_number  VARCHAR(30) NOT NULL UNIQUE,  -- RCF- series
    amount          NUMERIC(10,2) NOT NULL,
    payment_mode    VARCHAR(20) NOT NULL,
    transaction_ref VARCHAR(100),
    authorised_by   VARCHAR(20) NOT NULL,   -- senior user_id from Auth service
    processed_by    VARCHAR(20) NOT NULL,   -- cashier user_id from Auth service
    refund_date     DATE NOT NULL,
    notes           TEXT,
    created_at      TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | student_id | obligation_id | votehead_id | receipt_number | amount | payment_mode | authorised_by |
|---|---|---|---|---|---|---|---|
| RFD001 | S12347 | OBL004 | VH003 | RCF-2025-0001 | 1,000.00 | mpesa | USR002 |

**Note:** S12347 cleared OBL004 in full (37,000 paid, status = cleared). Programme complete. Finance admin (USR002) authorises the Caution Fee refund of 1,000. Cashier (USR001) processes it. Separate RCF- receipt issued. This is not a reversal — it is a deliberate outgoing payment back to the student.

## 10 audit_log

Every financial action writes one row here — payment posted, reversal created, refund processed, fee structure edited, votehead reordered, enrollment created. old_value and new_value are JSONB snapshots of the record before and after the change, enabling full reconstruction of any record's history.

SQL: audit_log

```
CREATE TABLE audit_log (
    id              VARCHAR(20) PRIMARY KEY,
    user_id         VARCHAR(20) NOT NULL,   -- from Auth service
    action          VARCHAR(60) NOT NULL,
    -- PAYMENT_POSTED | PAYMENT_REVERSED | ENROLLMENT_CREATED
    -- FEE_STRUCTURE_EDITED | VOTEHEAD_REORDERED | REFUND_PROCESSED
    entity_type     VARCHAR(40) NOT NULL,
    -- payment | enrollment | fee_structure | votehead | refund
    entity_id       VARCHAR(20) NOT NULL,
    old_value       JSONB,
    new_value       JSONB,
    ip_address      VARCHAR(45),
    created_at      TIMESTAMP DEFAULT NOW()
);
```

**Example data:**

| id | user_id | action | entity_type | entity_id | created_at |
|---|---|---|---|---|---|
| AUD001 | USR001 | ENROLLMENT_CREATED | enrollment | ENR001 | 2025-01-06 08:30 |
| AUD002 | USR001 | PAYMENT_POSTED | payment | PMT001 | 2025-01-15 09:42 |
| AUD003 | USR001 | PAYMENT_POSTED | payment | PMT002 | 2025-01-20 14:10 |
| AUD004 | USR002 | PAYMENT_REVERSED | payment | PMT006 | 2025-01-22 11:15 |
| AUD005 | USR002 | REFUND_PROCESSED | refund | RFD001 | 2025-06-15 10:00 |
| AUD006 | USR002 | FEE_STRUCTURE_EDITED | fee_structure | FS001 | 2025-01-04 14:00 |
| AUD007 | USR001 | VOTEHEAD_REORDERED | votehead | VH005 | 2025-01-03 09:00 |

# Schema Summary

10 tables. No intake table. No obligation lines. Enrollment date drives fee resolution. Payment distributions written at receipt time.

| # | Table | Purpose |
|---|---|---|
| 01 | `modules` | Course codes, names, levels, module numbers |
| 02 | `voteheads` | Fee categories — priority, one-time, refundable flags |
| 03 | `fee_structures` | Named fee config per module + category + date range |
| 04 | `fee_structure_lines` | Votehead amounts per structure — the waterfall template |
| 05 | `module_enrollments` | Billing trigger — fee_structure_id locked at enrollment |
| 06 | `fee_obligations` | Live balance per enrollment — total_paid, total_balance |
| 07 | `payments` | Immutable transaction records including reversals |
| 08 | `payment_distributions` | Votehead breakdown — written at receipt generation only |
| 09 | `refunds` | Programme-completion refunds (Caution etc.) |
| 10 | `audit_log` | Every financial action with JSONB before/after snapshots |

## Key Rules

- fee_structures references modules via module_id — no course/level/module columns anywhere else.
- module_enrollments locks fee_structure_id at enrollment time. Never changes after.
- fee_obligations updates total_paid and total_balance on payment posting. Nothing else happens at payment time.
- payment_distributions is written only at receipt generation — full waterfall replay at that point.
- Reversals live in payments as separate rows (is_reversal = true, reversal_of = original id). Original row untouched.
- Refunds are separate from reversals — different receipt series (RCF-), different table, different trigger.
- audit_log receives one row for every financial action — no exceptions.
- student_id and course_code are plain strings referencing external microservices — no DB-level FK constraints across service boundaries.